Destructuring, Spread & Rest

9 min read · Updated 17 Jul 2026 · Topic 19 of 23

Destructuring unpacks values out of arrays and objects into named variables in a single expression, while the spread and rest syntax (`...` — the same three dots, playing opposite roles) either *expands* a collection into individual elements or *collects* individual elements into a collection. These three features show up constantly in modern JS: React state updates, default-options merging, and pulling fields out of API responses all lean on them, so interviewers use them to check whether you actually write idiomatic ES6+ or just recognize it. The subtle points — shallow copies, when a default fires, destructuring `undefined` — are exactly where the trick questions live.

Array destructuring

You bind by **position**. Extra names get `undefined`; you can supply defaults, skip elements with holes, and nest to match structure.

JS Copy

```
const [a, b] = [1, 2];
console.log(a, b); // ⇒ 1 2

const [x = 10, y = 20] = [5];
console.log(x, y); // ⇒ 5 20  (x from array, y from default)

const [, , third] = ['a', 'b', 'c'];
console.log(third); // ⇒ c  (first two positions skipped)

const [[m, n], o] = [[1, 2], 3];
console.log(m, n, o); // ⇒ 1 2 3  (nested)
```

The classic party trick is **swapping two variables with no temp** — destructuring evaluates the right-hand array first, then assigns:

JS Copy

```
let p = 1, q = 2;
[p, q] = [q, p];
console.log(p, q); // ⇒ 2 1
```

Object destructuring

Here you bind by **key name**, not position, so order is irrelevant. You can rename (`key: newName`), give defaults, nest, and even use a computed (dynamic) key.

JS Copy

```
const user = { id: 7, name: 'Ada', role: 'admin' };

const { name: userName, role } = user;   // rename name → userName
console.log(userName, role); // ⇒ Ada admin

const { theme = 'light' } = {};           // default when key absent
console.log(theme); // ⇒ light

const { data: { items, page } } = { data: { items: [1, 2], page: 2 } }; //
console.log(items, page); // ⇒ [ 1, 2 ] 2

const key = 'score';
const { [key]: score } = { score: 99 };   // computed key
console.log(score); // ⇒ 99
```

Note that `data: { items, page }` does **not** create a `data` variable — the `data:` part is a path to descend into, and only `items` and `page` become bindings.

Destructuring in function parameters

Destructuring a parameter object turns positional arguments into a self-documenting "named arguments" call. Add `= {}` on the whole parameter so calling with no argument

doesn't throw.

JS Copy

```
function greet({ name, greeting = 'Hi' } = {}) {
  return `${greeting}, ${name}`;
}
console.log(greet({ name: 'Sam' })); // ⇒ Hi, Sam
console.log(greet());                // ⇒ Hi, undefined  (no throw, thank
```

Spread `...` — expanding a collection

Spread takes an iterable (or object) and *spreads* its elements into a new array, object, or argument list. It is the idiomatic way to concatenate, shallow-clone, and pass an array as individual arguments.

JS Copy

```
const c1 = [1, 2], c2 = [3, 4];
console.log([...c1, ...c2]); // ⇒ [ 1, 2, 3, 4 ]  (concat)

const cloneArr = [...c1];
console.log(cloneArr, cloneArr === c1); // ⇒ [ 1, 2 ] false  (new array)

console.log(Math.max(...[3, 1, 9, 4])); // ⇒ 9  (spread as arguments)
```

Object spread copies **own enumerable** properties. Later keys win, which makes merge-with-override a one-liner:

JS Copy

```
const base = { a: 1, b: 2 };
const merged = { ...base, b: 5, c: 3 };
console.log(merged); // ⇒ { a: 1, b: 5, c: 3 }  (b overridden, c added)
```

Rest `...` — collecting the leftovers

Rest is the mirror image: on the **left** of a destructuring, or as the **last function parameter**, `...` gathers whatever remains into a fresh array (or object).

JS Copy

```
const [head, ...tail] = [1, 2, 3, 4];
console.log(head, tail); // ⇒ 1 [ 2, 3, 4 ]

const user = { id: 7, name: 'Ada', role: 'admin' };
const { id, ...restProps } = user;
console.log(id, restProps); // ⇒ 7 { name: 'Ada', role: 'admin' }  (id re
```

Rest parameters replace the old `arguments` object. Unlike `arguments`, rest is a **real array** (so `.map` / `.reduce` work directly), only holds the parameters you didn't name, and works in arrow functions — which have no `arguments` at all.

JS Copy

```
function sum(...nums) {
  return nums.reduce((t, n) => t + n, 0);
}
console.log(sum(1, 2, 3, 4)); // ⇒ 10

function old() {
  return Array.isArray(arguments);
}
console.log(old(1, 2)); // ⇒ false  (arguments is array-LIKE, not a real
```

Spread vs rest — same token, opposite jobs

They read identically but are distinguished by **position**: `...` on the right-hand side (in a literal or call) is spread; `...` on the left-hand side (in a binding target or parameter list) is rest.

Role	Expands one collection into many elements	Collects many elements into one c
Where it appears	RHS: array/object literal, function *call*	LHS: destructuring pattern, functio
Example	`f( ... arr)`, `[ ... a, ... b]`, `{ ... o }`	`const [x, ... rest]`, `function`
Produces	Individual values	A new array (params/array) or obje
Count limit	Any number, anywhere	Exactly one, must be **last**

The shallow-copy caveat

Both spread cloning and object destructuring copy only **one level deep**. Top-level primitives are duplicated, but a nested object is copied *by reference* — the clone and the original still point at the same inner object.

JS Copy

```
const orig = { name: 'A', prefs: { dark: true } };
const copy = { ...orig };
copy.name = 'B';          // top-level: independent
copy.prefs.dark = false;  // nested: SHARED reference

console.log(orig.name);       // ⇒ A      (unchanged)
console.log(orig.prefs.dark); // ⇒ false  (mutated through the copy!)
```

When you need true isolation — nested state, config trees — reach for a **deep clone**. Modern Node/browsers have `structuredClone`:

JS Copy

```
const deep = structuredClone(orig);
deep.prefs.dark = true;
```

`JSON.parse(JSON.stringify(obj))` is the old fallback but drops functions, `undefined`, `Date` becomes a string, and it throws on cycles — prefer `structuredClone` when available.

Practical interview uses

Default options merge — spread user overrides over defaults so callers only specify what changes:

```js
function makeRequest(opts = {}) {
  const config = { method: 'GET', cache: true, ...opts };
  return config;
}
console.log(makeRequest({ method: 'POST' })); // ⇒ { method: 'POST', cach
```

Immutable state update — the React/Redux pattern: never mutate, spread the old state and overwrite the changed field:

```js
const state = { count: 0, user: { name: 'Ada' } };
const next = { ...state, count: state.count + 1 };
console.log(next.count, state.count); // ⇒ 1 0  (original untouched)
```

Extracting from API responses — nested destructuring with rename and defaults pulls exactly the fields you need and names them for local use: `const { data: { user: { name = 'Guest' } } } = response;`.

Common interview questions

Q: Swap two variables without a temporary variable. Use array destructuring — the RHS array is fully evaluated before assignment, so `[a, b] = [b, a]` swaps cleanly with

Q: Does `{ ...obj }` give you a deep copy? No — it is a **shallow** copy. Top-level properties are copied, but nested objects and arrays remain shared references, so mutating a nested field on the copy mutates the original too. Use `structuredClone` (or a library like lodash `cloneDeep`) for a deep copy.

Q: Difference between rest parameters and the `arguments` object? Rest (`...args`) is a genuine `Array` with all array methods, contains only the arguments not captured by named parameters, and is available in arrow functions. `arguments` is array-*like* (has `length`/indices but no `.map`), always holds every argument, and does not exist in arrow functions. Rest is the modern replacement.

Q: How do you clone an array and find its max in one expression each? Clone: `[...arr]`. Max: `Math.max(...arr)` — spread turns the array into the individual arguments `Math.max` expects (it takes numbers, not an array).

Q: What does `const { a, ...rest } = obj` do? It binds `a` to `obj.a` and collects all *remaining* own enumerable properties into a new object `rest`. It is the standard immutable way to "remove a key" — you get everything except `a` without mutating `obj`.

Gotchas / traps

Shallow copy shares nested state. As shown above, `{ ...orig }` copies `prefs` by reference, so `copy.prefs.dark = false` silently mutates `orig.prefs.dark`. This is the single most common destructuring/spread bug in interviews.

Destructuring `null` or `undefined` throws. You cannot read properties off nullish values, so the pattern throws a `TypeError` before any default can apply:

JS Copy

```
try {
  const { foo } = undefined;
} catch (e) {
```

Guard with a default on the source: `const { foo } = obj ?? {}` or `function f({ foo } = {})`.

A default fires only on `undefined`, never on `null`. Defaults trigger when the value is *strictly* `undefined` — `null`, `0`, `''`, and `false` are all "present" and pass straight through:

JS Copy

```
const { val = 'fallback' } = { val: null };
console.log(val); // ⇒ null  (null is NOT undefined, so no default)

const { val2 = 'fallback' } = { val2: undefined };
console.log(val2); // ⇒ fallback  (undefined triggers the default)
```

If you want `null` to also fall back, use the nullish coalescing operator explicitly: `const val = obj.val ?? 'fallback'`.

Array holes vs object rename confusion. `const [, , x] = arr` skips by position (commas matter); `const { a: x } = obj` renames by key. Mixing them up — expecting `{ 2: x }` to grab the third element — is a common slip since arrays are objects but destructure positionally when using array pattern syntax.

QUICK CHECK — no marks, just for you.

1. What does this log?

JAVASCRIPT Copy

```
const { data: { items, page } } = { data: { items: [1, 2], page: 2 } };
console.log(items, page);
```

- ◯ undefined 2

- [ ] `[ 1, 2 ] 2`
- [ ] `[ 1, 2 ] undefined`

Check answer

2. What does this log?

JAVASCRIPT Copy

```
const { val = 'fallback' } = { val: null };
const { val2 = 'fallback' } = { val2: undefined };
console.log(val, val2);
```

- [ ] `null fallback`
- [ ] `fallback fallback`
- [ ] `null undefined`
- [ ] `fallback null`

Check answer

3. What does this log?

JAVASCRIPT Copy

```
const orig = { name: 'A', prefs: { dark: true } };
const copy = { ...orig };
copy.name = 'B';
copy.prefs.dark = false;
console.log(orig.name, orig.prefs.dark);
```

A false

B false

B true

Check answer

Previous
Callbacks, Generators & Iterators

Next
Map, Set, WeakMap & WeakSet

LEARN MORE

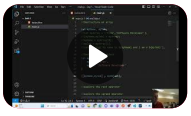
ARTICLES & DOCS

- Destructuring assignment (MDN)
  MDN
- Spread syntax (MDN)
  MDN

VIDEOS



Destructuring, spread & rest
YouTube



Destructuring and rest/spread
YouTube



Destructuring, spread & rest syntaxes
YouTube

Hand-picked references for this lesson.

JavaScript Interview Kit